Experiment – 22
Gaussian Mixture Model

Aim: To demonstrate Gaussian Mixture Model using Python

Definition: A Gaussian Mixture Model (GMM) is a probabilistic model that assumes data points are generated from a mixture of several Gaussian distributions with unknown parameters.

Concept:

- **Summary:** GMM is used for clustering and density estimation. It models data as a combination of multiple Gaussian distributions, each representing a different cluster.
- **Advantages:**
    - Can model complex data distributions.
    - Provides soft clustering, allowing data points to belong to multiple clusters.
    - Flexible in terms of the shape and size of clusters.
- **Disadvantages:**
    - Sensitive to the initial parameters.
    - Can converge to local optima.
    - Requires the number of components to be specified.

- **Uses:**
    - Image processing.
    - Anomaly detection.
    - Speech recognition.

- **Components:**
    - Mean vector (μ) for each Gaussian component.
    - Covariance matrix (Σ) for each Gaussian component.
    - Mixing coefficients (π) representing the weight of each component.

- **Challenges:**

- Determining the optimal number of components.
- Computationally intensive for large datasets.

Program:

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn import datasets

from sklearn.mixture import GaussianMixture

from sklearn.decomposition import PCA

from sklearn.metrics import silhouette_score, confusion_matrix

import seaborn as sns


# Load the Iris dataset

iris = datasets.load_iris()

X = iris.data

y_true = iris.target


# Reduce dimensions to 2 for visualization

pca = PCA(n_components=2)

X_reduced = pca.fit_transform(X)


# Fit GMM to the data

gmm = GaussianMixture(n_components=3, random_state=42)

gmm.fit(X_reduced)

labels = gmm.predict(X_reduced)
```

```
# Calculate silhouette score

sil_score = silhouette_score(X_reduced, labels)

print(f'Silhouette Score: {sil_score:.2f}')


# Plot the results

plt.figure(figsize=(10, 6))

scatter = plt.scatter(X_reduced[:, 0], X_reduced[:, 1],
c=labels, cmap='viridis', s=50)

plt.title('GMM Clustering on Iris Dataset')

plt.xlabel('PCA Component 1')

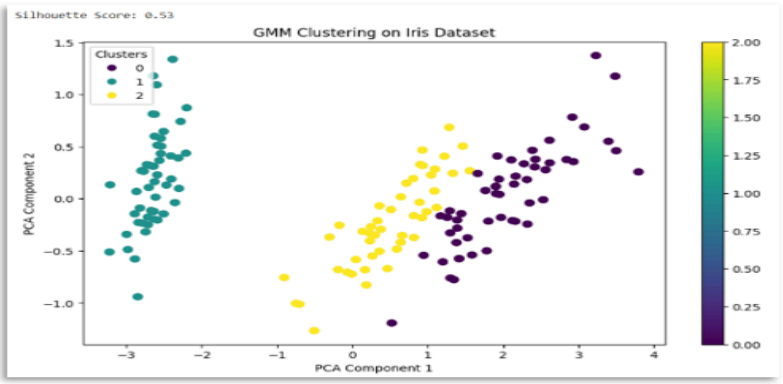
plt.ylabel('PCA Component 2')

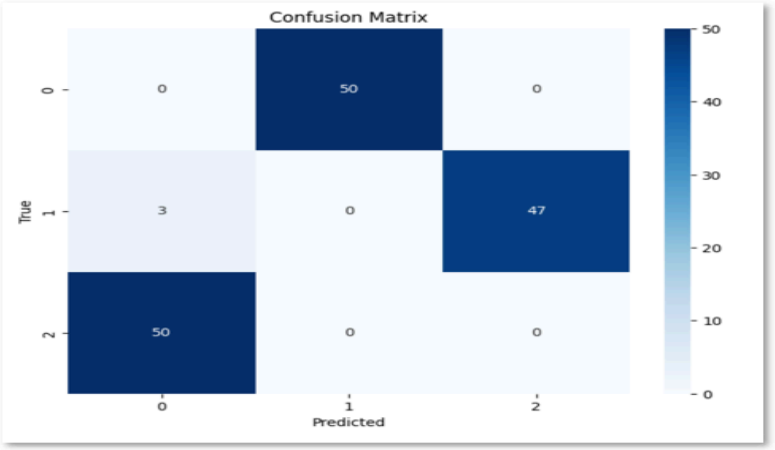
plt.legend(*scatter.legend_elements(), title="Clusters")

plt.colorbar()

plt.show()
```

Output:





Conclusion:

This code applies Gaussian Mixture Model (GMM) clustering to the Iris dataset, reducing its dimensions with PCA for visualization. It evaluates the clustering quality using the silhouette score and visualizes the clustered data in 2D.

Experiment – 23
Bayesian Gaussian Mixture Model

Aim: To demonstrate Gaussian Mixture Model using Python

Definition:
A Bayesian Gaussian Mixture Model (BGMM) is a probabilistic model that represents data as a mixture of multiple Gaussian distributions, with the parameters estimated through Bayesian inference, incorporating prior knowledge about the distribution of these parameters.

Concept:
The BGMM is an extension of the Gaussian Mixture Model (GMM), incorporating a Bayesian approach to prevent overfitting by introducing priors on the model parameters. It assumes that data is generated from several Gaussian distributions, each with its mean and variance, but allows for uncertainty in the parameters. BGMM adjusts the number of components automatically based on the data.

- **Advantages**: Handles uncertainty in model parameters, avoids overfitting with small data sets, automatically determines the number of clusters.
- **Disadvantages**: Computationally expensive, requires careful selection of priors.
- **Uses**: Clustering, density estimation, anomaly detection, speaker recognition, image segmentation.
- **Components**: Gaussian distributions, prior distributions, variational inference, mixture weights, and responsibilities (posterior probabilities).
- **Challenges**: High computational complexity, sensitivity to prior selection, model convergence issues.

Program:

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn import datasets

from sklearn.mixture import BayesianGaussianMixture

from sklearn.decomposition import PCA
```

```
from sklearn.metrics import silhouette_score, confusion_matrix

import seaborn as sns


# Load the Iris dataset

iris = datasets.load_iris()

X = iris.data

y_true = iris.target


# Reduce dimensions to 2 for visualization

pca = PCA(n_components=2)

X_reduced = pca.fit_transform(X)


# Fit BGMM to the data

bgmm = BayesianGaussianMixture(n_components=10,
random_state=42)

bgmm.fit(X_reduced)

labels = bgmm.predict(X_reduced)


# Calculate silhouette score

sil_score = silhouette_score(X_reduced, labels)

print(f'Silhouette Score: {sil_score:.2f}')


# Plot the results

plt.figure(figsize=(10, 6))

scatter = plt.scatter(X_reduced[:, 0], X_reduced[:, 1],
c=labels, cmap='viridis', s=50)
```

```
plt.title('BGMM Clustering on Iris Dataset')

plt.xlabel('PCA Component 1')

plt.ylabel('PCA Component 2')

plt.legend(*scatter.legend_elements(), title="Clusters")

plt.colorbar()

plt.show()

# Confusion matrix

cm = confusion_matrix(y_true, labels)

plt.figure(figsize=(8, 6))

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
xticklabels=np.unique(labels), yticklabels=np.unique(y_true))

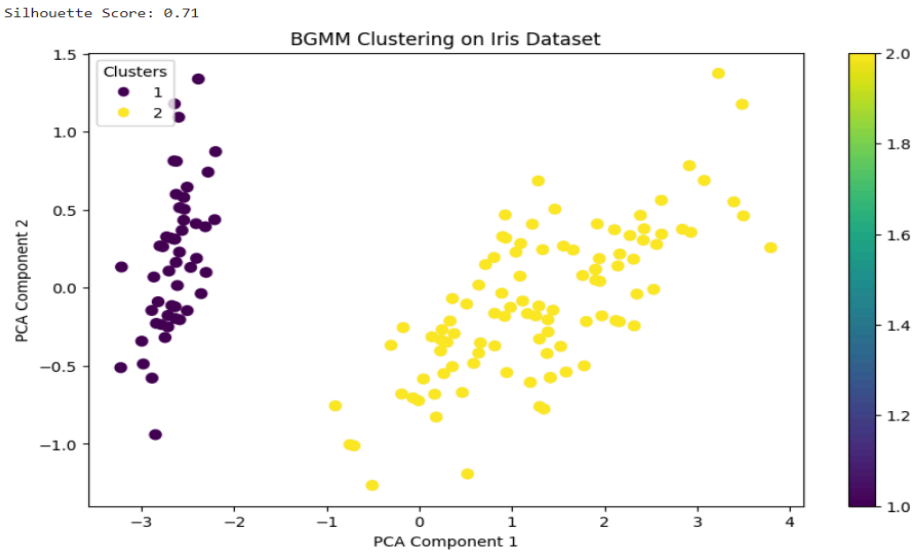
plt.title('Confusion Matrix')

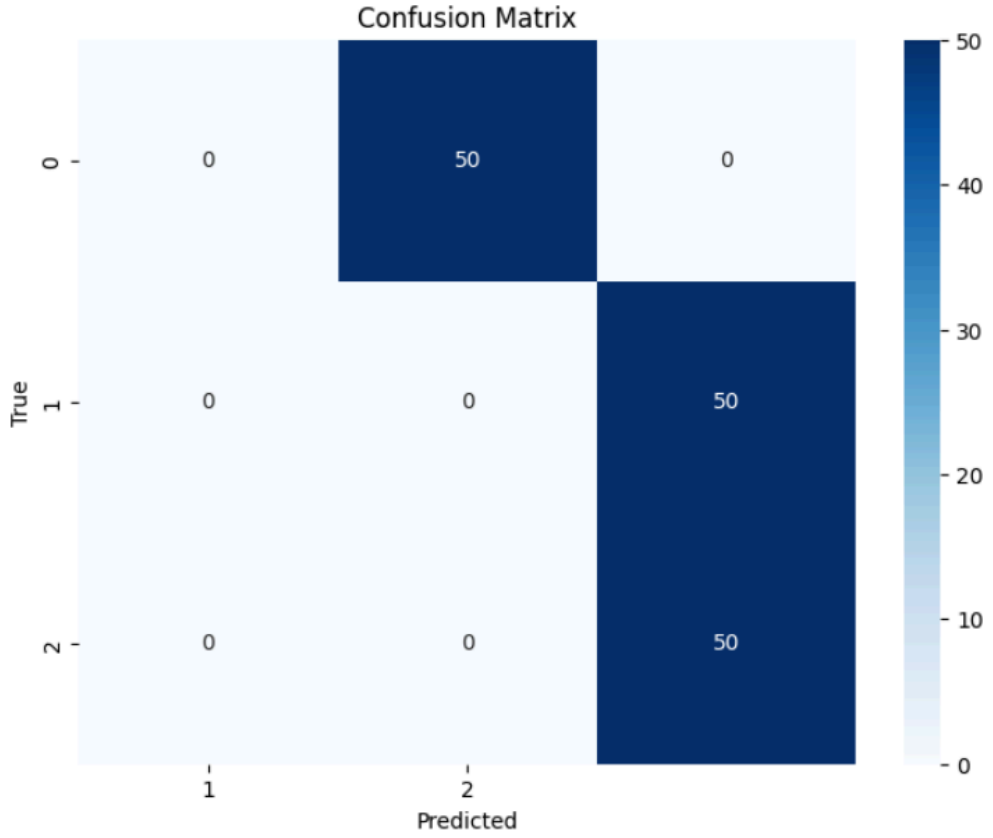
plt.xlabel('Predicted')

plt.ylabel('True')

plt.show()
```

Output:





Conclusion:

This code applies Bayesian Gaussian Mixture Model (BGMM) clustering to the Iris dataset and reduces its dimensions with PCA for visualization. It evaluates the clustering with a silhouette score and visualizes both the cluster assignments and confusion matrix, showing the relationship between predicted and true labels.